AGENDA 3

UTILIZANDO MICROSOFT MAUI PARA DESENVOLVIMENTO MOBILE

GEEaD - Grupo de Estudos de Educação a Distância

Centro de Educação Tecnológica Paula Souza

GOVERNO DO ESTADO DE SÃO PAULO

EIXO TECNOLÓGICO DE INFORMAÇÃO E COMUNICAÇÃO

CURSO TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

PROGRAMAÇÃO MOBILE II

Expediente

Autores:

KELLY CRISTIANE DE OLIVEIRA DAL POZZO

TIAGO ANTONIO DA SILVA

Revisão Técnica:

GUILHERME HENRIQUE GIROLLI

São Paulo – SP, 2024

Avançando os estudos de .NET MAUI com SQLite, nesta agenda vamos inserir o primeiro registro usando os recursos estudados e desenvolvidos nas agendas anteriores. Na primeira agenda criamos o projeto e a classe Model, na agenda passada implementamos toda a classe responsável pelas interações com o SQLite e nesta agenda vamos construir a interface gráfica necessária para preencher os dados e salvar no SQLite.

Para iniciar defina no aplicativo qual será o arquivo que armazenará os dados do SQLite, conforme App.xaml.cs:

```
public partial class App : Application
{
    static SQLiteDatabaseHelper _db;

    5 referências
    public static SQLiteDatabaseHelper Db
    {
        get
        {
            if(_db == null)
            {
                string path = Path.Combine(
                    Environment.GetFolderPath(
                        Environment.SpecialFolder.LocalApplicationData),
                    "banco_sqlite_compras.db3");

                _db = new SQLiteDatabaseHelper(path);
            }

            return _db;
        }
    }
```

Note que o trecho de código contém a definição de banco_sqlite_compras.db3. e vejamos a seguir o significado de cada trecho do código-fonte.

Singleton

Singleton é um padrão de projetos para a classe SQLiteDatabaseHelper, com o objetivo de garantir que apenas uma instância do banco de dados seja criada e usada ao longo da aplicação. Onde:

- static SQLiteDatabaseHelper _db

  O campo _db é estático, o que significa que ele pertence à classe, e não a uma instância específica dela.

Isso garante que o campo seja compartilhado entre todas as instâncias da classe. Já o **SQLiteDatabaseHelper_db** armazenará a instância única do objeto SQLiteDatabaseHelper, que gerencia a conexão com o banco de dados SQLite. A propriedade estática Db fornece acesso à instância do banco de dados. A ideia aqui é que o código da aplicação sempre use a mesma instância de SQLiteDatabaseHelper (padrão Singleton).

- if (_db == null)

  Verifica se a instância de _db ainda não foi criada. Caso seja null, significa que a instância ainda não foi inicializada, e o código dentro do if é executado.
- Path.Combine(...):

  Gera o caminho do arquivo do banco de dados. O método Path. Combine combina diferentes partes do caminho de forma que o código funcione corretamente em diferentes sistemas operacionais (por exemplo, Windows, macOS, Android).
- Environment.GetFolderPath(Environment.SpecialFolder.LocalApplicationData):

  Retorna o caminho para a pasta de dados locais da aplicação, que é um diretório usado para armazenar dados persistentes da aplicação.
- "banco_sqlite_compras.db3":

  Este é o nome do arquivo do banco de dados SQLite. O arquivo será criado ou aberto nesse local.
- db = new SQLiteDatabaseHelper(path);:

  Cria uma nova instância da classe SQLiteDatabaseHelper, passando o caminho do banco de dados. Esse é o ponto onde o banco de dados é inicializado. A instância criada será armazenada na variável _db.

Após a criação da instância (ou caso ela já exista), o método retorna a instância de _db.

Se a instância já tiver sido criada anteriormente, a verificação if (_db == null) não entrará no bloco de código e o código simplesmente retornará a instância existente de _db.

Como Funciona?

Quando a aplicação precisar acessar o banco de dados, ela chama a propriedade Db:

SQLiteDatabaseHelper db = App.Db;

O getter da propriedade Db verifica se a instância de _db já foi criada:

Se não foi criada (primeira vez), ele cria a instância e a inicializa com o caminho do banco de dados.

Vantagens do Padrão Singleton

Uso Eficiente de Recursos: O padrão Singleton garante que apenas uma instância do banco de dados seja criada, economizando recursos (como memória e tempo de processamento) ao evitar a criação de múltiplas instâncias do banco de dados.

Centralização do Acesso: A propriedade Db fornece um ponto centralizado de acesso ao banco de dados. Isso facilita a manutenção e o controle da instância do banco de dados em toda a aplicação.

Facilidade de Acesso: O padrão permite o acesso fácil e direto ao banco de dados a partir de qualquer lugar do código da aplicação, sem a necessidade de passar a instância explicitamente entre os métodos e classes.

Arquivo db3 do SQLite

O arquivo com a extensão .db3 é um arquivo de banco de dados utilizado pelo SQLite, O SQLite é popular devido à sua simplicidade, pequeno tamanho, e o fato de que ele armazena todos os dados em um único arquivo, ao contrário de outros SGBDs, que frequentemente utilizam múltiplos arquivos e requerem servidores de banco de dados separados. Vejamos as características do arquivo:

- **Formato do Arquivo SQLite:**

  O arquivo .db3 é um arquivo binário que contém todas as tabelas, índices, dados e esquemas de um banco de dados SQLite.

  O SQLite, por ser um banco de dados embutido, armazena todos os dados em um único arquivo, que pode ser facilmente movido ou copiado entre sistemas.

- **Extensão .db3:**

  A extensão .db3 não é obrigatória, pois o SQLite pode usar qualquer nome de arquivo (geralmente .db ou .sqlite), mas o .db3 é frequentemente utilizado em algumas convenções para identificar especificamente bancos de dados SQLite.

  O nome do arquivo pode ser qualquer string válida, e a extensão .db3 é apenas uma convenção que pode ser usada para indicar que o arquivo contém um banco de dados SQLite.

- **Conteúdo do Arquivo .db3:**

  O arquivo contém uma estrutura interna do banco de dados que inclui:

  - ✓ **Tabelas:** Armazenam dados em linhas e colunas.
  - ✓ **Índices:** Melhoram a performance de consultas.
  - ✓ **Triggers e Views:** Definem comportamentos automáticos e visualizações de dados.
  - ✓ **Dados:** Os valores reais armazenados nas tabelas.

- **Armazenamento e Acesso:**

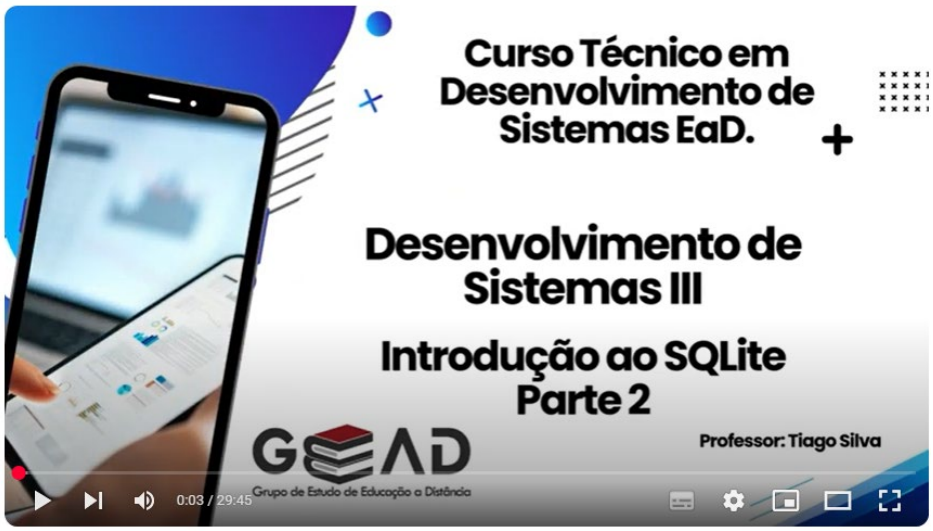
  O arquivo .db3 pode ser armazenado em diversos tipos de dispositivos, como discos locais, servidores ou sistemas de arquivos em nuvem, e pode ser acessado por qualquer aplicação que use SQLite.

  O SQLite oferece suporte a leitura e escrita concorrente. No entanto, para evitar conflitos de escrita, o SQLite utiliza um mecanismo de bloqueio no arquivo de banco de dados.

- **Portabilidade:**

  Uma das vantagens de usar o SQLite é a portabilidade. O banco de dados SQLite é independente de plataforma, o que significa que o arquivo .db3 pode ser copiado ou transferido entre diferentes sistemas operacionais (Windows, macOS, Linux) sem a necessidade de ajustes ou configurações extras. Isso é especialmente útil para aplicativos móveis e desktop que precisam de um banco de dados embutido e portátil, como aplicativos desenvolvidos em .NET MAUI

Assista o vídeo apresentando o passo a passo para criação do aplicativo:



Disponível em: https://www.youtube.com/watch?v=cAd28OMf7PA

CÓDIGO-FONTE DA AGENDA

https://github.com/tiagotas/MauiAppMinhasCompras/tree/9cdf2f376e5b896cf6ecc87ae6e2a63b86f268ab